

ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group XX: [names and student IDs]

2026-07-15

Contents

Authorship and contribution statement	2
Abstract	2
Reproducible setup	2
Shared helper functions	3
1 Problem 1. Prediction design and leakage control	5
1.1 1A. Target and predictors	5
1.2 1B. Split and train-only preprocessing	6
1.3 1C. Training-data exploration	6
2 Problem 2. OLS, ridge, and lasso	6
2.1 2A. OLS baseline	6
2.2 2B. Ridge	6
2.3 2C. Lasso	10
2.4 2D. Training-based comparison and locked core model	11
2.5 2E. Perturbation or stability check	11
3 Problem 3. Mathematical mechanisms	11
3.1 3A. Ridge and conditioning	11
3.2 3B. Lasso optimality and soft thresholding	11
3.3 3C. Connect algebra to evidence	11

4	Problem 4. Elastic net and a neural-feature bridge	11
4.1	4A. Research question and source map	11
4.2	4B. Elastic net on original predictors	12
4.3	4C. Fixed ReLU features	12
4.4	4D. Locked declaration and final holdout evaluation	12
4.5	4E. Deep-learning connection and limitations	12
5	Problem 5. Report quality and reproducibility	12
5.1	5A. Reproduction record	12
5.2	5B. Recommendation and limitations	13
5.3	5C. AI verification record	13
	Preflight and session information	13

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in `Group04_AI_Log.csv`, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	[Work]	[Check]
Nguyen Khang Thinh	24125104	[Work]	[Check]
Nguyen Van Tinh	24125106	[Work]	[Check]
Nguyen Le Quoc Huy	24125100	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}
```

```

}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)

group_number <- as.integer(params$group_number)
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)

```

Shared helper functions

```

find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {

```

```

x <- as.matrix(x)
center <- colMeans(x)
centered <- sweep(x, 2L, center, "-")
scale <- sqrt(colMeans(centered^2))
keep <- is.finite(scale) & scale > tol
if (!any(keep)) stop("No nonconstant training predictor remains.")
list(
  center = center[keep],
  scale = scale[keep],
  keep = keep,
  dropped = colnames(x)[!keep]
)
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

```

```

}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}

```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```

config <- list(
  file = "prostate.csv",
  response = "lpsa",
  excluded = "lpsa"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}

```

Define the prediction task and justify every exclusion.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]

foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

Report seeds, dimensions, scaling, folds, and leakage safeguards.

1.3 1C. Training-data exploration

```
# Use analysis_data[train_id, ] only. Add focused plots and summaries.
```

Interpret multicollinearity, one anomaly, and the question regularization will address.

2 Problem 2. OLS, ridge, and lasso

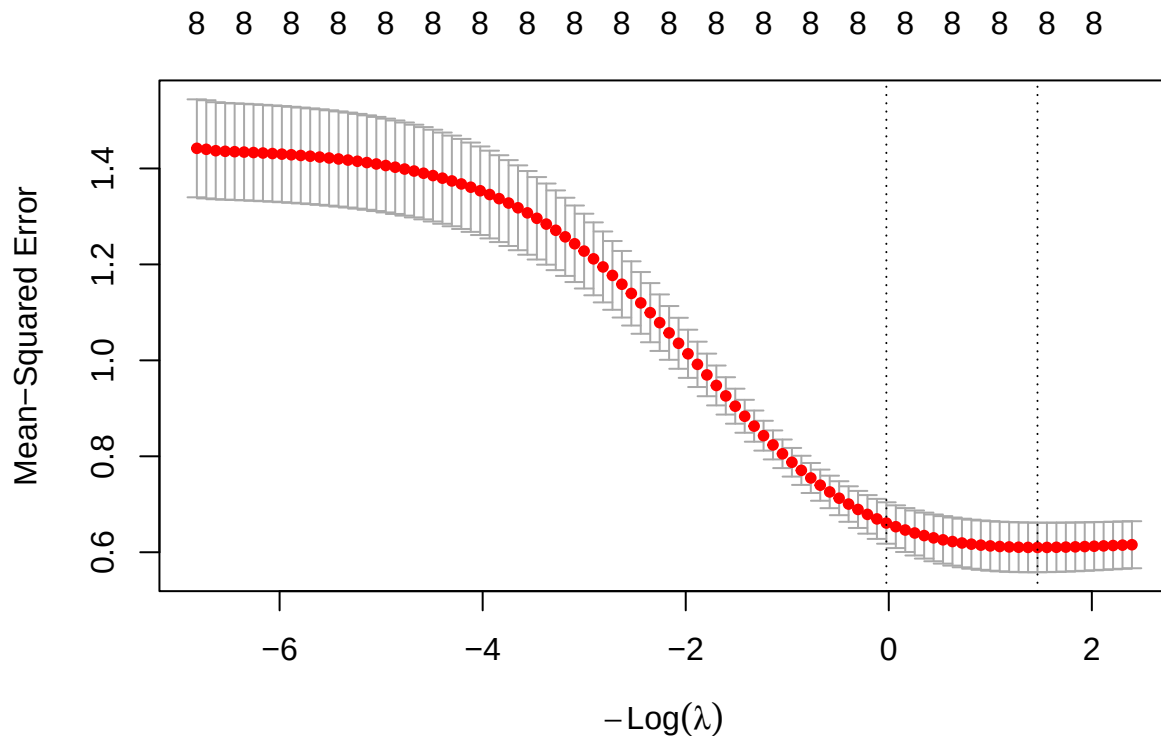
2.1 2A. OLS baseline

```
# Fit OLS on x_train and y_train. Do not access y_test here.
ols <- lm(y_train ~ x_train)
```

2.2 2B. Ridge

2.2.1 Cross-validation curve:

```
ridge <- fit_cv_glmnet(x = x_train, y = y_train, alpha=0, foldid = foldid)
plot(ridge)
```



The mean cross-validation MSE decreases as $-\text{Log}(\lambda)$ increases (equivalently, as λ decreases), suggesting that reducing the regularization strength improves predictive performance. The left dashed line indicates λ_{1se} , which selects a more regularized model with prediction error within one standard error of the minimum, while the right dashed line indicates λ_{min} , which yields the lowest cross-validation error.

2.2.2 Lambda min:

```
g <- crossprod(x_train)
eigen <- eigen(g)$value

lambda <- ridge$lambda.min
cat("Lambda min: ",lambda, "\n")
```

```
## Lambda min: 0.2311277
```

```
cat("MSE: ", ridge$cvm[ridge$lambda == lambda], "\n")
```

```
## MSE: 0.6100275
```

```
cat("Condition number:", (max(eigen) + lambda)/(min(eigen) + lambda), "\n")
```

```
## Condition number: 20.22951
```

With $\lambda_{min} \approx 0.23$, the 5-fold cross-validation MSE reaches its minimum value of $MSE_{min} \approx 0.61$, indicating the best predictive performance among the candidate values of λ . However, the corresponding Ridge model still has a relatively large condition number, suggesting that the model remains sensitive to multicollinearity and may not be sufficiently stable.

2.2.3 Lambda 1se:

```
g <- crossprod(x_train)
eigen <- eigen(g)$value

lambda <- ridge$lambda.1se
cat("Lambda 1se: ", lambda, "\n")
```

```
## Lambda 1se: 1.024039
```

```
cat("MSE: ", ridge$cvm[ridge$lambda == lambda], "\n")
```

```
## MSE: 0.6609706
```

```
cat("Condition number:", (max(eigen) + lambda)/(min(eigen) + lambda), "\n")
```

```
## Condition number: 19.06877
```

Although the one-standard-error rule selected a larger penalty $\lambda_{1se} \approx 1.02$, the condition number decreased only slightly from 20.2 to 19.07, indicating minimal improvement in numerical stability. In contrast, the cross-validation MSE increased from 0.61 to 0.66. Therefore, the additional regularization provided by λ_{1se} offers little practical benefit while reducing predictive performance.

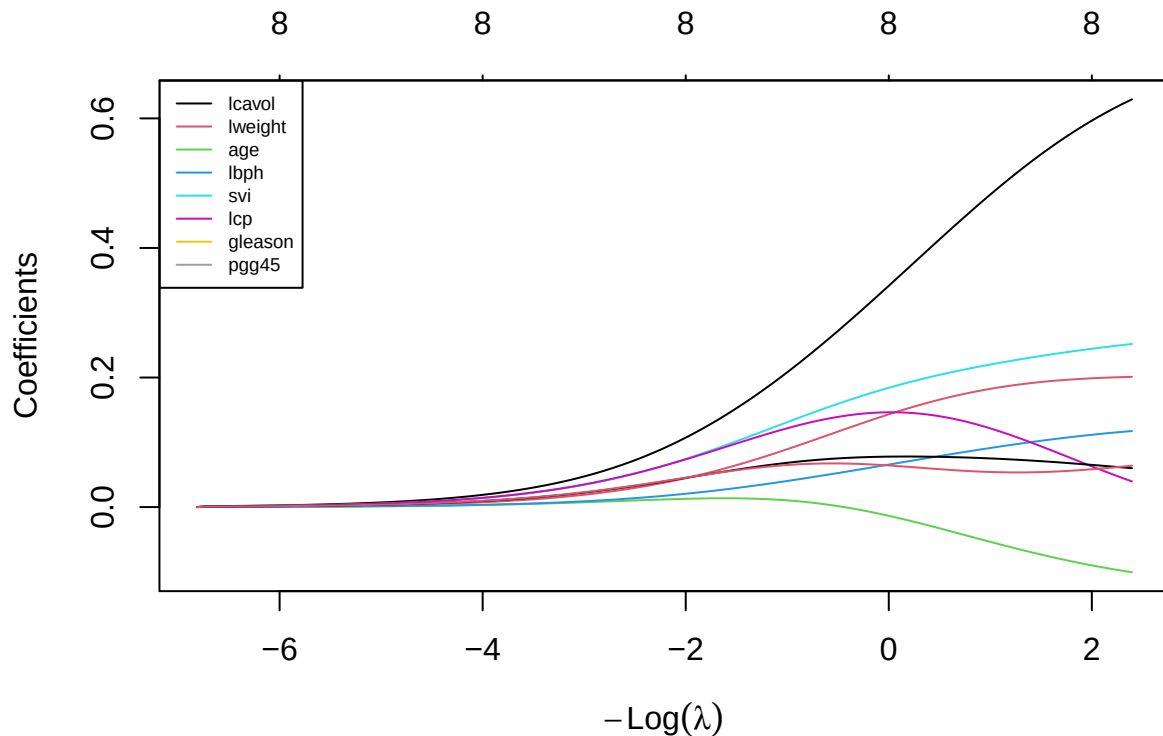
2.2.4 Coefficients:

```
data.frame(
  OLS = as.numeric(coef(ols)),
  Lambda_Min = as.numeric(coef(ridge, "lambda.min")),
  Lambda_Max = as.numeric(coef(ridge, "lambda.1se")),
  row.names = names(ols$coefficients)
)
```


	OLS	Lambda_Min	Lambda_Max
## (Intercept)	2.47569468	2.47569468	2.47569468
## x_trainlcavol	0.72282252	0.54055005	0.33822482
## x_trainlweight	0.20222141	0.19253291	0.14197627
## x_trainage	-0.13046518	-0.07201800	-0.01262288
## x_trainlbph	0.13361911	0.10166159	0.06498686
## x_trainsvi	0.27494937	0.23239051	0.18307566
## x_trainlcp	-0.04125506	0.09593321	0.14631601
## x_traingleason	0.03648450	0.07123914	0.07784505
## x_trainpgg45	0.09450125	0.05388195	0.06447679

Compared with the OLS model, Ridge regression shrinks most regression coefficients toward zero, reducing their magnitudes through the regularization penalty. This shrinkage helps alleviate the effects of multicollinearity while retaining all predictors in the model. Most coefficients exhibit the expected behavior by moving closer to zero as the regularization strength increases. However, two coefficients show notable exceptions. The coefficient of **lcp** changes from a small negative value in the OLS model to positive values under Ridge regression. In addition, the coefficient of **gleason** increases in magnitude, moving further away from zero rather than shrinking like the other coefficients. These changes suggest that the effects of **lcp** and **gleason** are influenced by multicollinearity with other predictors. As Ridge simultaneously estimates all coefficients under a penalty, it redistributes the effects among correlated variables, causing some coefficients to increase in magnitude or even change sign despite the overall shrinkage trend.

```
n_vars <- nrow(ridge$glmnet.fit$beta)
plot(ridge$glmnet.fit)
legend("topleft",
      legend = rownames(ridge$glmnet.fit$beta),
      col = 1:n_vars,
      lty = 1,
      cex = 0.6)
```



As λ increases, all regression coefficients are progressively shrunk toward zero, demonstrating the regularization effect of Ridge regression. Conversely, as λ decreases, the coefficients gradually move away from zero toward their OLS estimates. The coefficient of **lcavol** remains the largest throughout the regularization path, indicating that it has the strongest association with the response variable. The coefficients of **svi** and **lweight** also increase substantially as the regularization weakens, suggesting relatively strong effects on the response. In contrast, the coefficients of **gleason**, **lbph**, and **pgg45** remain relatively small across the entire path, indicating weaker contributions to the model. The coefficient of **age** remains negative throughout the regularization path, reflecting a consistent negative relationship with the response variable. Notably, the coefficient of **lcp** exhibits non-monotonic behavior, first increasing and then decreasing as λ changes, suggesting that its estimated effect is sensitive to multicollinearity with other predictors.

To address multicollinearity, Ridge impose an L_2 penalty on the regression coefficients. Rather than assigning a large coefficient to one correlated predictor and a small coefficient to another, Ridge shrinks the coefficients simultaneously and redistributes their effects among the correlated variables. Consequently, the coefficient estimates become more stable and less sensitive to small changes in the data.

2.3 2C. Lasso

```
# Use fit_cv_glmnet(..., alpha = 1, foldid = foldid).
# Report lambda.min, lambda.1se, paths, and nonzero predictors.
```

2.4 2D. Training-based comparison and locked core model

Build a comparison from training/CV quantities only.

Core model nominated before holdout evaluation: [method and reason]

2.5 2E. Perturbation or stability check

Prediction before running the check: [prediction]

Implement one allowed training-only stability check.

Observed result and explanation: [result]

3 Problem 3. Mathematical mechanisms

3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

*# Use x_{train} and the fitted ridge λ_{min} .
`safe_condition_numbers(x_train, lambda = ...)`*

3.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

3.3 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

Research direction: [weight decay / sparse weights / pruning / dropout]

Claim	Exact primary source and locator	What it establishes	What it does not establish
[Claim]	[Citation, section/page/equation]	[Support]	[Boundary]

4.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)

# Fit one cv.glmnet object per alpha with the same foldid.
# Select alpha and lambda using training CV only.
```

4.3 4C. Fixed ReLU features

```
relu <- function(z) pmax(z, 0)
M <- 200L

# TODO:
# 1. Generate A and bias once from seeds$features using the stated distributions.
# 2. Apply the same A and bias to x_train and x_test.
# 3. Fit a scaler to H_train only and apply it to H_test.
# 4. Fit ridge, lasso, and elastic net on H_train with the shared foldid.
```

State which weights are fixed, which are learned, and how active features are counted.

4.4 4D. Locked declaration and final holdout evaluation

Locked before viewing y_test: [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```
# This must be the first analysis chunk that uses y_test for evaluation.
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

4.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.

5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 26.04 LTS
##
## Matrix products: default
## BLAS: /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p-r0.3.32.so; LAPACK version
##
## locale:
## [1] LC_CTYPE=en_US.UTF-8 LC_NUMERIC=C
```

```

## [3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
## [5] LC_MONETARY=en_US.UTF-8   LC_MESSAGES=en_US.UTF-8
## [7] LC_PAPER=en_US.UTF-8     LC_NAME=C
## [9] LC_ADDRESS=C              LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Asia/Ho_Chi_Minh
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] knitr_1.51      broom_1.0.13    glmnet_5.0      Matrix_1.7-4
## [5] lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0   dplyr_1.2.1
## [9] purrr_1.2.2     readr_2.2.0     tidyr_1.3.2     tibble_3.3.1
## [13] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4    shape_1.4.6.1    stringi_1.8.7     lattice_0.22-9
## [5] hms_1.1.4         digest_0.6.39    magrittr_2.0.5     timechange_0.4.0
## [9] evaluate_1.0.5    grid_4.5.2       RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0     foreach_1.5.2    backports_1.5.1    survival_3.8-6
## [17] scales_1.4.0      codetools_0.2-20 cli_3.6.6          rlang_1.2.0
## [21] splines_4.5.2     withr_3.0.2      yaml_2.3.12        otel_0.2.0
## [25] tools_4.5.2       tzdb_0.5.0       vctrs_0.7.3        R6_2.6.1
## [29] lifecycle_1.0.5   pkgconfig_2.0.3  pillar_1.11.1      gtable_0.3.6
## [33] glue_1.8.1        Rcpp_1.1.2       xfun_0.58           tidyselect_1.2.1
## [37] rstudioapi_0.18.0 farver_2.1.2      htmltools_0.5.9    rmarkdown_2.31
## [41] compiler_4.5.2    S7_0.2.2

```